

Investment Strategy Detail - Technical Documentation

Generated: 22 January 2026 **Purpose:** Document the source, triggers, thresholds, and data flows for investment account features

Table of Contents

- 1. [Diversification Insights](#)
- 2. [Rebalancing Status](#)
- 3. [Total Fees](#)
- 4. [Annualised Return Figures](#)
- 5. [Monte Carlo Simulation](#)
- 6. [API Endpoints Summary](#)
- 7. [Empty Account Scenarios](#)
- 8. [UI Color Palette and Styling](#)

1. Diversification Insights

Source Files

Component	File Path
Backend Service	<code>app/Services/Investment/DiversificationAnalyzer.php</code>
Vue Component	<code>resources/js/components/Investment/DiversificationTab.vue</code>
API Service	<code>resources/js/services/diversificationService.js</code>

How They Are Triggered

- 1. `DiversificationTab.vue` mounts and calls `loadData()`
- 2. Component watches `accountId` prop for changes and reloads data
- 3. Vue service makes API call: `GET /investment/accounts/{accountId}/diversification`
- 4. Backend `DiversificationAnalyzer.analyze()` processes holdings

HHI (Herfindahl-Hirschman Index) Calculation

The HHI is a standard measure of market concentration adapted to measure portfolio diversification.

Actual code from `DiversificationAnalyzer.php` lines 50-68:

```
public function calculateHHI(Collection $holdings): float
{
    if ($holdings->isEmpty()) {
        return 0.0;
    }
}
```

```

$totalValue = $holdings->sum('current_value');
if ($totalValue <= 0) {
    return 0.0;
}

$hhi = 0.0;
foreach ($holdings as $holding) {
    $weight = ($holding->current_value ?? 0) / $totalValue;
    $hhi += $weight * $weight;
}

return round($hhi, 4);
}

```

Formula:

$$HHI = \sum (\text{holding_value} / \text{total_portfolio_value})^2$$

Step-by-step example:

1. Portfolio has 3 holdings: £10,000, £6,000, £4,000 (total = £20,000)
2. Calculate weights: 0.50, 0.30, 0.20
3. Square each weight: 0.25, 0.09, 0.04
4. Sum the squares: HHI = 0.38

Range: 0 (infinitely diversified) to 1 (single holding)

HHI Classification Thresholds

From [DiversificationAnalyzer.php](#) lines 73-79:

```

public function getHHILabel(float $hhi): string
{
    return match (true) {
        $hhi < 0.15 => 'Well Diversified',
        $hhi <= 0.25 => 'Moderate Concentration',
        default => 'High Concentration',
    };
}

```

HHI Value	Classification
< 0.15	Well Diversified
0.15 - 0.25	Moderate Concentration
> 0.25	High Concentration

Diversification Score Calculation

From `DiversificationAnalyzer.php` lines 257-294:

Base score starts at 100, then penalties/bonuses applied:

```
public function calculateDiversificationScore(float $hhi, array
$concentration, array $assetClassBreakdown): int
{
    $score = 100;

    // HHI penalty (0-40 points)
    if ($hhi >= 0.5) {
        $score -= 40;
    } elseif ($hhi >= 0.25) {
        $score -= 25;
    } elseif ($hhi >= 0.15) {
        $score -= 10;
    }

    // Concentration penalties (0-30 points)
    if ($concentration['top_holding_percent'] > 40) {
        $score -= 20;
    } elseif ($concentration['top_holding_percent'] > 25) {
        $score -= 10;
    }

    if ($concentration['top_3_holdings_percent'] > 80) {
        $score -= 10;
    } elseif ($concentration['top_3_holdings_percent'] > 60) {
        $score -= 5;
    }

    // Asset class diversity bonus/penalty (0-30 points)
    $classesUsed = collect($assetClassBreakdown)->filter(fn ($v) => $v >
0)->count();
    if ($classesUsed >= 4) {
        $score += 10;
    } elseif ($classesUsed === 1) {
        $score -= 20;
    } elseif ($classesUsed === 2) {
        $score -= 10;
    }

    return max(0, min(100, $score));
}
```

Condition	Score Change
HHI >= 0.5	-40 points

Condition	Score Change
HHI >= 0.25	-25 points
HHI >= 0.15	-10 points
Top holding > 40%	-20 points
Top holding > 25%	-10 points
Top 3 holdings > 80%	-10 points
Top 3 holdings > 60%	-5 points
Single asset class only	-20 points
Only 2 asset classes	-10 points
4+ asset classes	+10 points (bonus)

Score Labels

From [DiversificationAnalyzer.php](#) lines 299-306:

```
public function getScoreLabel(int $score): string
{
    return match (true) {
        $score >= 80 => 'Excellent',
        $score >= 60 => 'Good',
        $score >= 40 => 'Fair',
        default => 'Poor',
    };
}
```

Concentration Warnings

From [DiversificationAnalyzer.php](#) lines 127-153:

```
public function getConcentrationWarnings(array $concentration): array
{
    $warnings = [];

    if ($concentration['top_holding_percent'] > 25) {
        $warnings[] = [
            'type' => 'warning',
            'message' => 'Single holding exceeds 25% of portfolio –
consider reducing concentration',
        ];
    }

    if ($concentration['top_3_holdings_percent'] > 60) {
        $warnings[] = [
```

```
        'type' => 'warning',
        'message' => 'Top 3 holdings account for over 60% of
portfolio',
    ];
}

if ($concentration['holdings_over_10_percent'] > 3) {
    $warnings[] = [
        'type' => 'info',
        'message' => sprintf('%d holdings exceed 10%% each - monitor
concentration', ...),
    ];
}

return $warnings;
}
```

Condition	Warning Type
Single holding > 25%	warning
Top 3 holdings > 60%	warning
More than 3 holdings > 10% each	info

Asset Class Mapping

From `DiversificationAnalyzer.php` lines 12-23:

```
private const ASSET_CLASS_MAP = [
    'uk_equity' => 'equities',
    'us_equity' => 'equities',
    'international_equity' => 'equities',
    'equity' => 'equities',
    'bond' => 'bonds',
    'cash' => 'cash',
    'alternative' => 'alternatives',
    'property' => 'alternatives',
    'fund' => 'equities',
    'etf' => 'equities',
];
```

Risk Level Mapping

From `DiversificationAnalyzer.php` lines 35-44:

```
private const RISK_LEVEL_MAP = [
    'low' => 1,
    'lower_medium' => 2,
```

```
    'medium' => 3,
    'upper_medium' => 4,
    'high' => 5,
    'cautious' => 2,      // Legacy mapping
    'balanced' => 3,      // Legacy mapping
    'adventurous' => 4,   // Legacy mapping
  ];
```

Target Asset Allocations by Risk Level

From `DiversificationAnalyzer.php` lines 26-32.

Each allocation has a **±5% acceptable range** to allow flexibility and natural crossover between adjacent risk levels:

Level 1: Low

Asset Class	Target	Acceptable Range
Equities	10%	5% - 15%
Bonds	70%	65% - 75%
Cash	20%	15% - 25%
Alternatives	0%	0% - 5%

Level 2: Lower-Medium

Asset Class	Target	Acceptable Range
Equities	30%	25% - 35%
Bonds	55%	50% - 60%
Cash	10%	5% - 15%
Alternatives	5%	0% - 10%

Level 3: Medium

Asset Class	Target	Acceptable Range
Equities	50%	45% - 55%
Bonds	35%	30% - 40%
Cash	10%	5% - 15%
Alternatives	5%	0% - 10%

Level 4: Upper-Medium

Asset Class	Target	Acceptable Range
Equities	75%	70% - 80%
Bonds	15%	10% - 20%
Cash	5%	0% - 10%
Alternatives	5%	0% - 10%

Level 5: High

Asset Class	Target	Acceptable Range
Equities	90%	85% - 95%
Bonds	0%	0% - 5%
Cash	5%	0% - 10%
Alternatives	5%	0% - 10%

Summary Table

Numeric	String Level	Equities	Bonds	Cash	Alternatives
1	low	5-15%	65-75%	15-25%	0-5%
2	lower_medium	25-35%	50-60%	5-15%	0-10%
3	medium	45-55%	30-40%	5-15%	0-10%
4	upper_medium	70-80%	10-20%	0-10%	0-10%
5	high	85-95%	0-5%	0-10%	0-10%

Note: These ranges align with the default tolerance band of $\pm 5\%$ used in the rebalancing strategy (see Section 2).

Deviation Severity

From `DiversificationAnalyzer.php` lines 229-235:

```
public function getDeviationSeverity(float $deviation): string
{
    return match (true) {
        $deviation < 5 => 'aligned',
        $deviation <= 10 => 'minor',
        default => 'significant',
    };
}
```

2. Rebalancing Status

Overview

The rebalancing system monitors holding values within each account. As market movements change holding values, the system calculates the current asset allocation percentages and compares them to the acceptable ranges defined for the user's risk level (see Section 1).

Core principle: Only recommend rebalancing when an asset class allocation falls OUTSIDE its acceptable range.

How It Works

Step 1: Track Holding Values

For each account, the system tracks:

- Current value of each holding
- Total account value
- Asset class of each holding (equities, bonds, cash, alternatives)

Step 2: Calculate Current Asset Allocation

For each asset class:

$$\text{Asset Class \%} = (\text{Sum of holdings in that class} / \text{Total Account Value}) \times 100$$

Example:

- Account total: £100,000
- Equity holdings: £55,000 → 55%
- Bond holdings: £32,000 → 32%
- Cash: £8,000 → 8%
- Alternatives: £5,000 → 5%

Step 3: Compare to Acceptable Ranges

The current allocation is compared against the acceptable ranges for the user's risk level.

Example for Medium risk (Level 3):

Asset Class	Acceptable Range	Current	Status
Equities	45% - 55%	55%	✓ Within range
Bonds	30% - 40%	32%	✓ Within range
Cash	5% - 15%	8%	✓ Within range
Alternatives	0% - 10%	5%	✓ Within range

Result: No rebalancing needed - all allocations within acceptable ranges.

Step 4: Trigger Rebalancing (When Needed)

Rebalancing is only recommended when ONE OR MORE asset classes fall outside their acceptable range.

Example - Portfolio has drifted:

Asset Class	Acceptable Range	Current	Status
Equities	45% - 55%	62%	× ABOVE range (+7%)
Bonds	30% - 40%	25%	× BELOW range (-5%)
Cash	5% - 15%	8%	✓ Within range
Alternatives	0% - 10%	5%	✓ Within range

Result: Rebalancing recommended - equities and bonds are outside acceptable ranges.

Rebalancing Status Indicators

Status	Condition	Colour	Action
Aligned	All asset classes within acceptable ranges	Green	No action needed
Minor Drift	One asset class outside range by < 5%	Amber	Consider rebalancing
Significant Drift	One or more asset classes outside range by ≥ 5%	Red	Rebalancing recommended

Rebalancing Recommendations

When rebalancing is needed, the system calculates adjustments to bring allocations back to target:

For each out-of-range asset class:

 If current > max: Sell to reach target

 If current < min: Buy to reach target

Example adjustments for the drifted portfolio above (£100,000 total):

Asset Class	Current	Target	Action
Equities	£62,000 (62%)	£50,000 (50%)	Sell £12,000
Bonds	£25,000 (25%)	£35,000 (35%)	Buy £10,000
Cash	£8,000 (8%)	£10,000 (10%)	Buy £2,000
Alternatives	£5,000 (5%)	£5,000 (5%)	No change

Acceptable Ranges Reference

These are the ranges used for rebalancing decisions (from Section 1):

Risk Level	Equities	Bonds	Cash	Alternatives
Low	5-15%	65-75%	15-25%	0-5%
Lower-Medium	25-35%	50-60%	5-15%	0-10%
Medium	45-55%	30-40%	5-15%	0-10%
Upper-Medium	70-80%	10-20%	0-10%	0-10%
High	85-95%	0-5%	0-10%	0-10%

Source Files

Component	File Path
Drift Analyzer	app/Services/Investment/Rebalancing/DriftAnalyzer.php
Strategy Service	app/Services/Investment/Rebalancing/RebalancingStrategyService.php
Vue Component	resources/js/components/Investment/RebalancingActions.vue

3. Total Fees

Source Files

Component	File Path
Fee Analyzer	app/Services/Investment/FeeAnalyzer.php

Fee Calculation

From `FeeAnalyzer.php` lines 33-85:

```
public function calculateTotalFees(Collection $accounts, Collection $holdings): array
{
    $portfolioValue = $accounts->sum('current_value');

    // Calculate platform fees
    $platformFees = $accounts->sum(function ($account) {
        return $account->current_value * ($account->platform_fee_percent /
100);
    });

    // Calculate fund OCF (Ongoing Charges Figure)
    $fundFees = $holdings->sum(function ($holding) {
```

```

        return $holding->current_value * ($holding->ocf_percent / 100);
    });

    // Estimated transaction costs (simplified)
    $transactionCosts = $portfolioValue * 0.001; // 0.1% estimated

    $totalFees = $platformFees + $fundFees + $transactionCosts;
    $feeDragPercent = ($totalFees / $portfolioValue) * 100;
    // ...
}

```

Formula:

Total Annual Fees = Platform Fees + Fund OCF + Transaction Costs +
 Advisory Fees
 Fee Drag % = (Total Fees / Portfolio Value) × 100

Fee Components

1. Platform Fees

From [FeeAnalyzer.php](#) lines 350-375:

```

private function calculatePlatformFee(float $portfolioValue, string
$platformName): float
{
    $platformFees = match (strtolower($platformName)) {
        'vanguard' => $this->calculateTieredFee($portfolioValue, [
            [0, 250000, 0.0015],
            [250000, PHP_FLOAT_MAX, 0.00375],
        ]),
        'hargreaves lansdown', 'hl' => $this-
>calculateTieredFee($portfolioValue, [
            [0, 250000, 0.0045],
            [250000, 1000000, 0.0025],
            [1000000, PHP_FLOAT_MAX, 0.0010],
        ]),
        'aj bell' => $this->calculateCappedFee($portfolioValue, 0.0025,
3.50, 7.50),
        'interactive investor', 'ii' => 9.99 * 12, // Flat monthly fee
        'fidelity' => $this->calculateCappedFee($portfolioValue, 0.0035,
0, 45),
        'charles stanley direct' => $this-
>calculateTieredFee($portfolioValue, [
            [0, 50000, 0.0025],
            [50000, 500000, 0.0015],
            [500000, PHP_FLOAT_MAX, 0.0010],
        ]),
        default => $portfolioValue * 0.0030, // Industry average ~0.30%
    };
}

```

```
};

return $platformFees;
}
```

Platform	Fee Structure
Vanguard	0.15% (up to £250k), 0.375% (above)
Hargreaves Lansdown	0.45% (up to £250k), 0.25% (£250k-£1m), 0.10% (above)
AJ Bell	0.25% capped between £3.50-£7.50
Interactive Investor	£9.99/month (£119.88/year flat)
Fidelity	0.35% capped at £45
Charles Stanley Direct	0.25% / 0.15% / 0.10% tiered
Default	0.30% (industry average)

2. Fund OCF (Ongoing Charges Figure)

Calculation: `holding.current_value × (holding.ocf_percent / 100)`

Estimated OCF by Asset Type (if not provided):

From `FeeAnalyzer.php` lines 429-438:

```
private function estimateOCF(string $assetType): float
{
    return match ($assetType) {
        'index_fund', 'etf' => 0.001, // 0.10% for passive
        'active_fund' => 0.0075, // 0.75% for active
        'equity', 'stock' => 0.0, // No OCF for direct equities
        'bond' => 0.0005, // 0.05% for bond funds
        'alternative' => 0.015, // 1.5% for alternatives
        default => 0.005, // 0.50% default
    };
}
```

Asset Type	Estimated OCF
Index funds/ETFs	0.10%
Active funds	0.75%
Direct equities/stocks	0.00%
Bond funds	0.05%
Alternatives	1.50%

Asset Type	Estimated OCF
Default	0.50%

3. Transaction Costs

From `FeeAnalyzer.php` line 56:

```
$transactionCosts = $portfolioValue * 0.001; // 0.1% estimated
```

Estimation: $\text{portfolio_value} \times 0.001$ (0.1% based on assumed turnover)

4. Advisory Fees

If applicable: Uses `account.advisory_fee` field directly.

High-Fee Identification

Threshold Logic:

The high-fee threshold is **0.8%**, but **advisory fees are excluded** from the comparison. This is because advisory fees are a separate service cost that the user has consciously agreed to, and shouldn't trigger high-fee warnings.

Calculation:

```
Fees for threshold comparison = Total fees - Advisory fees  
High-fee flag triggered if: (Fees for threshold comparison) > 0.8%
```

Implementation:

```
public function identifyHighFeeHoldings(Collection $holdings, ?float  
$advisoryFee = null): array  
{  
    $highFeeThreshold = 0.8; // 0.8% is considered high (excluding  
    advisory fees)  
  
    $highFeeHoldings = $holdings->filter(function ($holding) use  
($highFeeThreshold, $advisoryFee) {  
        // Calculate fees excluding advisory fee for threshold comparison  
        $feesForComparison = $holding->ocf_percent;  
        if ($advisoryFee !== null) {  
            $feesForComparison = max(0, $holding->ocf_percent -  
$advisoryFee);  
        }  
        return $feesForComparison > $highFeeThreshold;  
    }->map(function ($holding) {
```

```
        return [
            'security_name' => $holding->security_name,
            'ocf_percent' => round($holding->ocf_percent, 4),
            'current_value' => round($holding->current_value, 2),
            'annual_cost' => round($holding->current_value * ($holding->ocf_percent / 100), 2),
            'recommendation' => 'Consider lower-cost alternative',
        ];
    });
    // ...
}
```

Threshold	Condition	Action
0.8%	Fees (excluding advisory) > 0.8%	Flagged as high-fee holding

Example:

Scenario	Total Fee	Advisory Fee	Fee for Comparison	Flagged?
No advisor	0.90%	0%	0.90%	✓ Yes (> 0.8%)
With advisor	1.20%	0.50%	0.70%	✗ No (< 0.8%)
With advisor, still high	1.50%	0.50%	1.00%	✓ Yes (> 0.8%)

Rationale:

- Advisory fees represent a conscious choice to pay for financial advice
- The high-fee threshold targets platform/fund costs that can often be reduced
- Users should not be warned about fees they've explicitly agreed to pay for advice

Fee Assessment Tiers

A simple tiered system based on total fees (excluding advisory fees):

Fee Range	Assessment	User Message
< 0.8%	Acceptable	No warning shown
0.8% - 1.0%	Higher than average	"Your fees are higher than average"
1.0% - 1.5%	High	"Your fees are high"
> 1.5%	Very high	"Your fees are much higher than average"

Implementation:

```
public function assessFeeLevel(float $totalFeePercent, ?float $advisoryFee = null): array
{
    // Exclude advisory fees from assessment
```

```
$feesForAssessment = $totalFeePercent;
if ($advisoryFee !== null) {
    $feesForAssessment = max(0, $totalFeePercent - $advisoryFee);
}

if ($feesForAssessment < 0.8) {
    return ['level' => 'acceptable', 'message' => null];
} elseif ($feesForAssessment <= 1.0) {
    return ['level' => 'higher_than_average', 'message' => 'Your fees
are higher than average'];
} elseif ($feesForAssessment <= 1.5) {
    return ['level' => 'high', 'message' => 'Your fees are high'];
} else {
    return ['level' => 'very_high', 'message' => 'Your fees are much
higher than average'];
}
}
```

Example scenarios:

Total Fee	Advisory Fee	Fee for Assessment	Result
0.60%	0%	0.60%	Acceptable (no warning)
0.95%	0%	0.95%	"Higher than average"
1.30%	0%	1.30%	"High"
1.80%	0%	1.80%	"Much higher than average"
1.30%	0.50%	0.80%	"Higher than average"
1.80%	0.50%	1.30%	"High"

4. Annualised Return Figures

Source Files

Component	File Path
Calculator	app/Services/Investment/PortfolioAnalyzer.php
Vue Display	resources/js/components/Investment/PortfolioOverview.vue

Calculation Method

From `PortfolioAnalyzer.php` lines 24-51:

```
public function calculateReturns(Collection $holdings): array
{
    $totalCostBasis = $holdings->sum('cost_basis');
    $totalCurrentValue = $holdings->sum('current_value');
```

```

    if ($totalCostBasis == 0) {
        return [
            'total_cost_basis' => 0.0,
            'total_current_value' => 0.0,
            'total_gain' => 0.0,
            'total_return_percent' => 0.0,
            'ytd_return' => 0.0,
            'one_year_return' => 0.0,
        ];
    }

    $totalGain = $totalCurrentValue - $totalCostBasis;
    $totalReturnPercent = ($totalGain / $totalCostBasis) * 100;

    return [
        'total_cost_basis' => round($totalCostBasis, 2),
        'total_current_value' => round($totalCurrentValue, 2),
        'total_gain' => round($totalGain, 2),
        'total_return_percent' => round($totalReturnPercent, 2),
        'ytd_return' => round($totalReturnPercent, 2), // Simplified
        'one_year_return' => round($totalReturnPercent, 2), // Simplified
    ];
}

```

Formulas:

Total Gain = Total Current Value - Total Cost Basis
 Total Return % = (Total Gain / Total Cost Basis) × 100

Note: Current implementation uses total return as simplified approximation. YTD and 1-year returns use the same value. In production, this would filter holdings by purchase date.

Data Sources

- `holdings.cost_basis` - Original purchase cost
- `holdings.current_value` - Current market value

Display Format

From `PortfolioOverview.vue`:

- Shows with +/- sign: `+2.34%` or `-1.50%`
- Colour-coded: Green if ≥ 0 , Red if < 0

5. Monte Carlo Simulation

Source Files

Component	File Path
Simulator	app/Services/Investment/MonteCarloSimulator.php
Projection Service	app/Services/Investment/InvestmentProjectionService.php
Risk Service	app/Services/Risk/RiskPreferenceService.php
Vue Component	resources/js/components/Investment/InvestmentProjectionChart.vue

Simulation Parameters

From `MonteCarloSimulator.php` lines 20-27:

```
public function simulate(  
    float $startValue,  
    float $monthlyContribution,  
    float $expectedReturn,  
    float $volatility,  
    int $years,  
    int $iterations = 1000  
): array
```

Parameter	Description	Source
<code>startValue</code>	Initial portfolio value	Account <code>current_value</code>
<code>monthlyContribution</code>	Monthly contribution	Account <code>monthly_contribution_amount</code>
<code>expectedReturn</code>	Expected annual return (decimal)	Risk profile typical return / 100
<code>volatility</code>	Annual volatility (decimal)	Risk profile volatility / 100
<code>years</code>	Projection period	5, 10, 20, 30 years
<code>iterations</code>	Simulation runs	1000 (from <code>InvestmentProjectionService</code> line 16)

Risk Level Configurations

From `RiskPreferenceService.php` lines 34-85:

```
private array $riskLevelConfigs = [  
    'low' => [  
        'level_numeric' => 1,  
        'display_name' => 'Low',  
        'asset_allocation' => ['equities' => 10, 'bonds' => 70, 'cash' =>  
20, 'alternatives' => 0],
```

```
        'expected_returns' => ['min' => 1.0, 'max' => 3.0, 'typical' =>
2.0],
        'volatility_percent' => 3.0,
    ],
    'lower_medium' => [
        'level_numeric' => 2,
        'display_name' => 'Lower-Medium',
        'asset_allocation' => ['equities' => 30, 'bonds' => 55, 'cash' =>
10, 'alternatives' => 5],
        'expected_returns' => ['min' => 2.0, 'max' => 4.5, 'typical' =>
3.5],
        'volatility_percent' => 6.0,
    ],
    'medium' => [
        'level_numeric' => 3,
        'display_name' => 'Medium',
        'asset_allocation' => ['equities' => 50, 'bonds' => 40, 'cash' =>
5, 'alternatives' => 5],
        'expected_returns' => ['min' => 3.5, 'max' => 6.5, 'typical' =>
5.0],
        'volatility_percent' => 10.0,
    ],
    'upper_medium' => [
        'level_numeric' => 4,
        'display_name' => 'Upper-Medium',
        'asset_allocation' => ['equities' => 75, 'bonds' => 20, 'cash' =>
0, 'alternatives' => 5],
        'expected_returns' => ['min' => 5.0, 'max' => 8.5, 'typical' =>
6.5],
        'volatility_percent' => 15.0,
    ],
    'high' => [
        'level_numeric' => 5,
        'display_name' => 'High',
        'asset_allocation' => ['equities' => 90, 'bonds' => 5, 'cash' =>
0, 'alternatives' => 5],
        'expected_returns' => ['min' => 6.0, 'max' => 12.0, 'typical' =>
8.0],
        'volatility_percent' => 20.0,
    ],
];
```

String Level	Display Name	Min Return	Typical Return	Max Return	Volatility
low	Low	1.0%	2.0%	3.0%	3.0%
lower_medium	Lower-Medium	2.0%	3.5%	4.5%	6.0%
medium	Medium	3.5%	5.0%	6.5%	10.0%
upper_medium	Upper-Medium	5.0%	6.5%	8.5%	15.0%
high	High	6.0%	8.0%	12.0%	20.0%

Note: Monte Carlo simulation uses the **typical** return value from the user's risk profile.

Monthly Conversion

From **MonteCarloSimulator.php** lines 29-30:

```
$monthlyReturn = $expectedReturn / 12;  
$monthlyVolatility = $volatility / sqrt(12);
```

Random Number Generation (Box-Muller Transform)

From **MonteCarloSimulator.php** lines 103-118:

```
public function generateNormalDistribution(float $mean, float $stdDev):  
float  
{  
    // Box-Muller transform  
    $u1 = mt_rand() / mt_getrandmax();  
    $u2 = mt_rand() / mt_getrandmax();  
  
    // Ensure u1 is not zero to avoid log(0)  
    $u1 = max($u1, 1e-10);  
  
    // Apply Box-Muller transform  
    $z0 = sqrt(-2.0 * log($u1)) * cos(2.0 * M_PI * $u2);  
  
    // Transform to desired mean and standard deviation  
    return $mean + ($z0 * $stdDev);  
}
```

Simulation Process

From **MonteCarloSimulator.php** lines 33-55:

```
for ($i = 0; $i < $iterations; $i++) {  
    $portfolioValue = $startValue;  
    $yearlyValues = [];  
  
    for ($month = 1; $month <= $totalMonths; $month++) {  
        // Generate random return using normal distribution  
        $randomReturn = $this->generateNormalDistribution($monthlyReturn,  
$monthlyVolatility);  
  
        // Apply return and add contribution  
        $portfolioValue = $portfolioValue * (1 + $randomReturn) +  
$monthlyContribution;  
  
        // Store value at end of each year
```

```

        if ($month % 12 == 0) {
            $yearlyValues[] = $portfolioValue;
        }
    }

    $results[] = [
        'final_value' => $portfolioValue,
        'yearly_values' => $yearlyValues,
    ];
}

```

Algorithm:

1. For each of 1000 iterations:
 - Start with initial portfolio value
 - For each month:
 - Generate random return using Box-Muller transform (normal distribution)
 - Apply: $\text{portfolioValue} = \text{portfolioValue} \times (1 + \text{randomReturn}) + \text{monthlyContribution}$
 - Store yearly values at end of each 12-month period

Percentile Calculation

From `MonteCarloSimulator.php` lines 126-153:

```

public function calculatePercentiles(array $sortedValues): array
{
    $count = count($sortedValues);
    $percentiles = [];

    foreach ([10, 25, 50, 75, 90] as $p) {
        $index = (int) ceil(($p / 100) * $count) - 1;
        $index = max(0, min($index, $count - 1));
        $value = round($sortedValues[$index], 2);
        $percentiles[] = [
            'percentile' => "{$p}th",
            'value' => $value,
        ];
    }

    return $percentiles;
}

```

Formula: $\text{Index} = \text{ceil}((\text{percentile} / 100) \times \text{count}) - 1$

Returns 5 percentile buckets: 10th, 25th, 50th (median), 75th, 90th

Probability Band Smoothing

From `InvestmentProjectionService.php` lines 268-286:

```
// Smooth transition for early years – blend with start value
// Year 1: 70% Monte Carlo, 30% start value
// Year 2: 90% Monte Carlo, 10% start value
// Year 3+: 100% Monte Carlo
$blendFactor = 1.0;
if ($yearIndex === 1) {
    $blendFactor = 0.7;
} elseif ($yearIndex === 2) {
    $blendFactor = 0.9;
}

$p5 = $this->blendValue($p5, $startValue, $blendFactor);
// ... same for other percentiles
```

Year	Weighting
Year 1	70% simulation + 30% start value
Year 2	90% simulation + 10% start value
Year 3+	100% simulation

Interpolated Percentiles

From `InvestmentProjectionService.php` lines 262-266:

```
// Interpolate 5th, 15th, 20th (same as Retirement)
$spread = $p25 - $p10;
$p5 = $p10 - ($spread * 0.33);
$p15 = $p10 + ($spread * 0.33);
$p20 = $p10 + ($spread * 0.67);
```

The 5th, 15th, and 20th percentiles are interpolated from the 10th and 25th.

Chart Display Labels

From `InvestmentProjectionChart.vue` lines 99-101:

```
const levelDisplay = this.formatRiskLevel(this.riskLevel);
const formattedReturn = Number(this.expectedReturn).toFixed(2);
return `Using ${levelDisplay} risk profile (${formattedReturn}% expected return)`;
```

Expected return is formatted to 2 decimal places.

Risk Level Determination from Return

From `InvestmentProjectionService.php` lines 370-386:

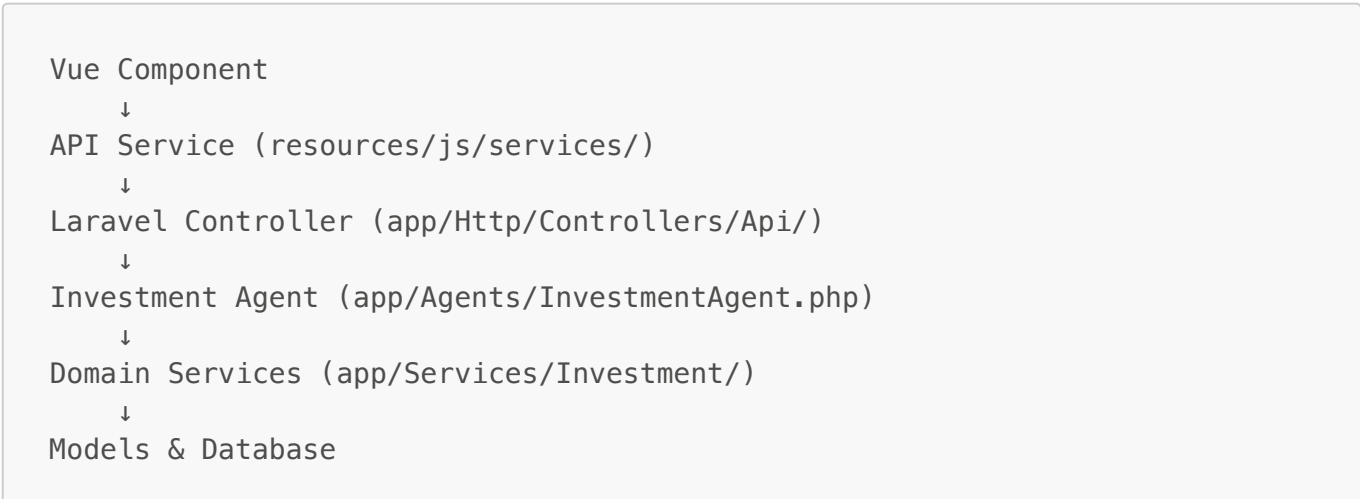
```
private function determineRiskLevel(float $typicalReturn): string
{
    if ($typicalReturn <= 2.5) {
        return 'low';
    }
    if ($typicalReturn <= 4.25) {
        return 'lower_medium';
    }
    if ($typicalReturn <= 5.75) {
        return 'medium';
    }
    if ($typicalReturn <= 7.25) {
        return 'upper_medium';
    }

    return 'high';
}
```

6. API Endpoints Summary

Feature	Endpoint	Method
Diversification Analysis	<code>/investment/accounts/{id}/diversification</code>	GET
Rebalancing Analysis	<code>/investment/accounts/{id}/rebalancing</code>	GET
Fee Analysis	<code>/investment</code> (included in response)	GET
Returns	<code>/investment</code> (included in response)	GET
Monte Carlo Projections	<code>/investment/projections</code>	GET
Recommendations	<code>/investment/recommendations</code>	GET

Architecture Flow



Example Flow for Diversification:

- 1. `DiversificationTab.vue` mounts
- 2. Calls `diversificationService.getAccountDiversification(accountId)`
- 3. API request: `GET /investment/accounts/{id}/diversification`
- 4. Controller calls `DiversificationAnalyzer.analyze()`
- 5. Analyzer processes holdings, calculates HHI, scores, warnings
- 6. Returns comprehensive analysis JSON
- 7. Vue component renders with score gauge, asset breakdown, recommendations

Key Configuration Locations

Configuration	Location
Risk level returns/volatility	<code>app/Services/Risk/RiskPreferenceService.php</code> lines 34-85
Target asset allocations	<code>app/Services/Investment/DiversificationAnalyzer.php</code> lines 26-32
Risk level mapping	<code>app/Services/Investment/DiversificationAnalyzer.php</code> lines 35-44
Platform fee rates	<code>app/Services/Investment/FeeAnalyzer.php</code> lines 353-375
Fee benchmarks	<code>app/Services/Investment/FeeAnalyzer.php</code> lines 486-514
Rebalancing thresholds	<code>app/Services/Investment/Rebalancing/RebalancingStrategyService.php</code> lines 411-440
Drift urgency thresholds	<code>app/Services/Investment/Rebalancing/DriftAnalyzer.php</code> lines 220-248
Monte Carlo iterations	<code>app/Services/Investment/InvestmentProjectionService.php</code> line 16
Projection periods	<code>app/Services/Investment/InvestmentProjectionService.php</code> line 14

7. Empty Account Scenarios

This section documents what happens when a user enters investment accounts without holdings or fee information.

Scenario 1: Investment Account with No Holdings (No Fees)

User action: Creates an investment account (e.g., ISA) with only:

- Provider name
- Account name
- Current value (e.g., £50,000)
- Account type

No holdings, no platform fee, no advisor fee entered.

Investment List View (**InvestmentList.vue**)

Account Card displays:

Element	Value Shown	Source
Account type badge	e.g., "Stocks & Shares ISA"	account.account_type
Provider name	e.g., "Vanguard"	account.provider
Account name	e.g., "General ISA"	account.account_name
Current Value	e.g., "£50,000"	account.current_value
YTD Return	Not shown	account.ytd_return is null
Risk Badge	Not shown	account.risk_preference is null
ISA Used (YTD)	"£0" (if ISA)	account.isa_subscription_current_year defaults to 0

Performance Section (right column):

The **Performance.vue** component displays:

- **Portfolio Projection chart** - YES, still runs Monte Carlo simulation
- **Current Portfolio value** - Shown (sum of all account values)
- **Projected Value (95%)** - Shown (based on default medium risk level)
- **Asset Allocation** - **NOT shown** (requires **analysis.allocation** which needs holdings)

Strategy Card:

If no risk profile exists, shows recommendation:

"Complete Your Risk Profile - Set up your risk profile to get personalised investment recommendations and target allocations."

If accounts exist but no holdings, shows recommendation:

"Add Your Holdings - Add your fund holdings to get detailed fee analysis, diversification scores, and tax efficiency recommendations."

Account Detail View (**InvestmentDetailInline.vue**)

Header Metrics:

Metric	Value Shown	Notes
Current Value	£50,000	From <code>account.current_value</code>
Monthly contribution	Not shown	Only shows if <code>estimatedMonthlyContribution > 0</code>
Annualised Return	"N/A"	<code>grossReturnPercent</code> returns null when no holdings
Net Return	Not shown	Conditional on <code>grossReturnPercent !== null</code>
Holdings count	"0"	<code>account.holdings.length</code>

Default Tab: Performance

Performance Tab (`AccountPerformancePanel.vue`)

Left Sidebar Cards:

Card	Content	Click Action
Diversification Insights	"Enter Holdings" link + "Add holdings to see diversification analysis"	Emits <code>add-holding</code> event
Rebalancing Status	"Enter Holdings" link + "Add holdings to get rebalancing strategies"	Emits <code>add-holding</code> event
Total Fees	"Add Fees" link + "Add fees to get fee optimisation strategies"	Goes to Fees tab

Chart Area:

Element	Displayed	Notes
Projected Value (95%) card	YES	Shows value based on Monte Carlo with default risk
Period selector	YES	5/10/20/30 years
Monte Carlo chart	YES	Runs simulation using <code>current_value</code> and default medium risk (5% return, 10% volatility)
Asset Allocation bar	NO	Requires <code>hasHoldings</code> to be true

Tax Status Card:

Element	Displayed
Tax Treatment	YES - shows based on account type (e.g., ISA)
Tax Items grid	YES - shows tax-free status for dividends, capital gains, etc.

Diversification Tab (DiversificationTab.vue)

Empty State Displayed:

 (amber icon)

No Holdings Recorded

Add holdings to this account to see diversification analysis.

[Add Holdings] (button)

Analysis Triggered: NO - `data.has_holdings` is false, so no HHI, score, or recommendations are calculated.

Fees Tab (AccountFeesPanel.vue)

Fee Summary Cards:

Card	Value	Notes
Platform Fee	0.00%	<code>account.platform_fee_percent</code> is null
Average Fund Fee (OCF)	0.00%	No holdings to calculate weighted average
Advisor Fee	0.00%	<code>account.advisor_fee_percent</code> is null
Total Annual Cost	0.00%	Sum of above

Annual Cost Breakdown:

Row	Value
Platform Fee (0.00%)	£0
Fund Fees - OCF (0.00%)	£0
Total Annual Cost	£0

Fund Fee Breakdown table: NOT shown - displays instead:

"No holdings data available for fee breakdown."

10-Year Fee Impact:

Metric	Value
Total Fees Over 10 Years	£0

Metric	Value
Projected Portfolio (Without Fees)	£81,445 (at 5% growth)
Projected Portfolio (With Fees)	£81,445
Fee Drag (Lost Growth)	£0

Scenario 2: Joint Investment Account with No Holdings + Monthly Contributions

User action: Creates a JOINT investment account with:

- Provider name, account name
- Current value: £100,000
- Ownership type: Joint
- Ownership percentage: 50%
- Monthly contribution: £500

No holdings, no fees.

Investment List View

Account Card displays:

Element	Value Shown
Joint badge	"Joint" (purple badge)
Account type badge	e.g., "General Investment"
Full Value	£100,000
Your Share (50%)	£50,000 (in purple text)

Account Detail View

Header Metrics:

Metric	Value Shown
Current Value	£100,000
Your 50% share	£50,000 (additional line in purple)
Monthly contribution	+£500 /month (shown in green)
Annualised Return	"N/A"
Holdings count	0

Performance Tab

Monte Carlo Projections:

The projection service calculates based on:

- Starting value: **User's share** (£50,000, not full value)
- Monthly contribution: £500 (applied to projections)
- Risk level: Default medium (5% return, 10% volatility)

Projected Value (95%) at 10 years:

Using default medium risk with £500/month contributions:

Starting: £50,000

Monthly: £500

Risk: medium (5% expected return)

Result: Monte Carlo simulation runs with contributions factored in

The chart shows probability bands expanding over time, with higher values than Scenario 1 due to ongoing contributions.

Scenario 3: Another Investment Account with No Holdings + Monthly Contributions

User action: Creates a second individual account with:

- Account type: SIPP
- Current value: £75,000
- Monthly contribution: £300

No holdings, no fees.

Portfolio-Level Impact

With multiple accounts, the system calculates aggregate portfolio metrics:

Portfolio Projection (**Performance.vue**):

Metric	Calculation
Total Portfolio Value	Sum of all account values (user's shares for joint)
Estimated Monthly Contribution	Sum of all account contributions

For example with both accounts:

- Account 1 (Joint): £50,000 (your share) + £500/month
- Account 2 (SIPP): £75,000 + £300/month
- **Portfolio Total:** £125,000 + £800/month

Portfolio Projection runs with:

- Combined starting value: £125,000
- Combined monthly contribution: £800
- Weighted risk level (defaults to medium if no risk profile set)

Recommendations Generated ([InvestmentAgent.php](#))

With accounts but no holdings:

Priority	Recommendation	Source
1	"Complete Your Risk Profile" (if not set)	Lines 118-126
2	"Add Your Holdings"	Lines 129-137

Text displayed:

"Add Your Holdings - Add your fund holdings to get detailed fee analysis, diversification scores, and tax efficiency recommendations. Click on your investment account and add your holdings."

Summary: What Analysis Is/Isn't Triggered

Analysis	Without Holdings	With Holdings
Monte Carlo Projection	✓ YES (uses account value + contributions)	✓ YES
Diversification Score	× NO (shows empty state)	✓ YES
HHI Calculation	× NO	✓ YES
Asset Allocation	× NO	✓ YES
Rebalancing Status	× NO (shows "Enter Holdings")	✓ YES
Fee Breakdown by Holding	× NO	✓ YES
Fee Summary Cards	✓ YES (shows 0% values)	✓ YES
10-Year Fee Impact	✓ YES (shows £0 impact)	✓ YES
Tax Status	✓ YES (based on account type)	✓ YES
Risk-based Recommendations	Partial (only "Add Holdings")	✓ Full recommendations

Key Files for Empty State Handling

Component	File	Empty State Logic
-----------	------	-------------------

Component	File	Empty State Logic
List Performance	Performance.vue lines 19-35	Shows "No Performance Data" if no accounts
Account Performance	AccountPerformancePanel.vue lines 21-24, 57-60, 131-134	Shows "Enter Holdings" links
Diversification	DiversificationTab.vue lines 21-30	Shows "No Holdings Recorded"
Fees	AccountFeesPanel.vue lines 110-113	Shows "No holdings data available"
Agent Recommendations	InvestmentAgent.php lines 129-137	Generates "Add Holdings" recommendation
Detail Header	InvestmentDetailInline.vue lines 67-81	Shows "N/A" for returns

User Journey: Empty Account to Full Analysis

- 1. **Create Account** → See account card with value, Monte Carlo projection runs
- 2. **Enter Holdings** → Unlock diversification analysis, HHI, asset allocation
- 3. **Add Fees** → See accurate fee breakdown, 10-year impact calculations
- 4. **Set Risk Profile** → Get personalised target allocations and rebalancing recommendations

Each step progressively unlocks more analysis features.

8. UI Color Palette and Styling

This section documents the color scheme used across investment detail view components. The palette avoids warning-signal colors (red, amber, yellow) to maintain a professional, calm interface.

Design Principles

- 1. **No red for negative states** - Red signals danger/warnings; use gray instead for negative returns
- 2. **No amber/yellow** - These create urgency; use blue or violet for elevated metrics
- 3. **Greens for positive** - Retained for good performance, well-diversified states
- 4. **Blues for informational** - Moderate states, elevated but not alarming metrics
- 5. **Violets for attention** - Replaces amber/red for items needing review (sparingly used)
- 6. **Grays for neutral** - Negative values, unknown states, disabled elements

Color Mapping by Component

InvestmentDetailInline.vue (Header Metrics)

Element	Color	Tailwind Class
Primary card (Current Value)	Light blue	bg-blue-50 border border-blue-200
Secondary cards	Light gray	bg-gray-50

Element	Color	Tailwind Class
Positive returns	Green	text-green-600
Negative returns	Gray	text-gray-600
ISA remaining (healthy)	Green	text-green-600
ISA remaining (low < £5k)	Blue	text-blue-600
ISA fully used	Gray	text-gray-600

AccountPerformancePanel.vue (Performance Tab)

Drift Status:

Status	Color	Tailwind Class
Aligned	Green	text-green-600
Minor Drift	Blue	text-blue-600
Significant Drift	Violet	text-violet-600

Fee Assessment:

Fee Level	Color	Tailwind Class
< 0.8% (Acceptable)	Green	text-green-600
0.8% - 1.5% (Elevated)	Blue	text-blue-600
> 1.5% (High)	Violet	text-violet-600

Tax Status Badges:

Status	Background	Tailwind Class
Exempt	Green	bg-green-500 text-white
Taxable	Slate	bg-slate-500 text-white
Deferred	Blue	bg-blue-500 text-white
Relief	Purple	bg-purple-500 text-white
Limit	Gray	bg-gray-500 text-white

Recommendation Cards:

Importance	Border Color	Tailwind Class
High priority	Violet	border-l-violet-500
Standard	Blue	border-l-blue-500

DiversificationTab.vue

Diversification Score:

Score Range	Color	Tailwind Class
>= 80 (Excellent)	Green	text-green-600
>= 60 (Good)	Blue	text-blue-600
>= 40 (Fair)	Violet	text-violet-600
< 40 (Poor)	Gray	text-gray-600

HHI Badges:

Classification	Background	Tailwind Class
Well Diversified	Green	bg-green-500 text-white
Moderate Concentration	Blue	bg-blue-500 text-white
High Concentration	Violet	bg-violet-500 text-white

Empty State:

Element	Color	Tailwind Class
Container	Light blue	bg-blue-50
Icon	Blue	text-blue-400
Button	Blue	bg-blue-600 hover:bg-blue-700

Warning Icons:

Severity	Color	Tailwind Class
Warning	Violet	text-violet-500
Info	Blue	text-blue-500

Header Card Layout

The header metrics cards match the Pension Detail View styling:

```
<div class="grid grid-cols-1 md:grid-cols-4 gap-4 mb-6">
  <!-- Primary card -->
  <div class="bg-blue-50 border border-blue-200 rounded-lg p-4">
    <p class="text-xs text-gray-500 uppercase tracking-wide">Current
Value</p>
    <p class="text-2xl font-bold text-gray-900">£50,000</p>
  </div>
```



```
<!-- Secondary cards -->
<div class="bg-gray-50 rounded-lg p-4">
  <p class="text-xs text-gray-500 uppercase tracking-wide">Metric</p>
  <p class="text-lg font-semibold text-gray-900">Value</p>
</div>
</div>
```

Key styling:

- 4-column grid on desktop (`md:grid-cols-4`)
- Primary card: `bg-blue-50 border border-blue-200`
- Secondary cards: `bg-gray-50`
- Label: `text-xs text-gray-500 uppercase tracking-wide`
- Primary value: `text-2xl font-bold`
- Secondary values: `text-lg font-semibold`

Files Updated

File	Changes
<code>resources/js/components/NetWorth/InvestmentDetailInline.vue</code>	Header card styling, return color methods
<code>resources/js/views/Investment/AccountPerformancePanel.vue</code>	Drift, fee, tax status, recommendation colors
<code>resources/js/components/Investment/DiversificationTab.vue</code>	Score, HHI, empty state, warning colors

Color Reference Quick Guide

Purpose	Avoid	Use Instead
Negative performance	<code>text-red-*</code>	<code>text-gray-600</code>
Elevated metrics	<code>text-amber-*</code> , <code>text-yellow-*</code>	<code>text-blue-600</code>
Needs attention	<code>text-red-*</code> , <code>bg-red-*</code>	<code>text-violet-600</code> , <code>bg-violet-*</code>
Positive states	-	<code>text-green-600</code> , <code>bg-green-*</code>
Neutral/unknown	-	<code>text-gray-*</code> , <code>bg-gray-*</code>
Informational	-	<code>text-blue-*</code> , <code>bg-blue-*</code>